FINE-TUNING TIẾT KIỆM THAM SỐ

LoRA Fine-tuning

Là gì · Hoạt động thế nào · Cài đặt ra sao

Dựa trên bài báo “LoRA: Low-Rank Adaptation of Large Language Models” — Hu và cộng sự, 2021

NỘI DUNG

Ba câu hỏi

01

LoRA fine-tuning là gì?

Vì sao full fine-tuning tốn kém, và LoRA thay đổi điều gì.

02

LoRA hoạt động thế nào?

Cập nhật hạng thấp, lượt forward, khởi tạo và gộp trọng số.

03

Cài đặt LoRA ra sao?

Một lớp PyTorch 15 dòng, fine-tune một model thật, và thư viện peft.

01 · LORA FINE-TUNING LÀ GÌ

Fine-tuning là dạy tiếp một model có sẵn

Pre-training

Hàng nghìn tỷ token. Học ngôn ngữ chung. Tốn hàng tháng tính toán — chỉ các lab lớn làm được.

Fine-tuning

Bộ dữ liệu nhỏ, chuyên biệt. Học tác vụ, định dạng hay văn phong của bạn. Huấn luyện tiếp trên chính các trọng số đó.

$$W_{mới} = W_{cũ} + \Delta W$$

Mỗi lớp của mạng nơ-ron là một ma trận trọng số W. Fine-tuning chính là học phần thay đổi ΔW.

Full fine-tuning rất tốn kém

~128 GB

Bộ nhớ GPU để full fine-tune một model 8 tỷ tham số bằng Adam ở fp32 — chưa tính activation.

Và mỗi tác vụ lại cần một bản sao đầy đủ 16 GB của model.

Lưu cho mỗi tham số	Byte
Trọng số (fp32)	4
Gradient	4
Adam: moment bậc 1	4
Adam: moment bậc 2	4
Tổng	**16**

LoRA đóng băng model đã pre-train và chỉ học một bản cập nhật nhỏ, có hạng thấp cho trọng số.

0,18%

trọng số của DistilGPT-2 được train trong demo

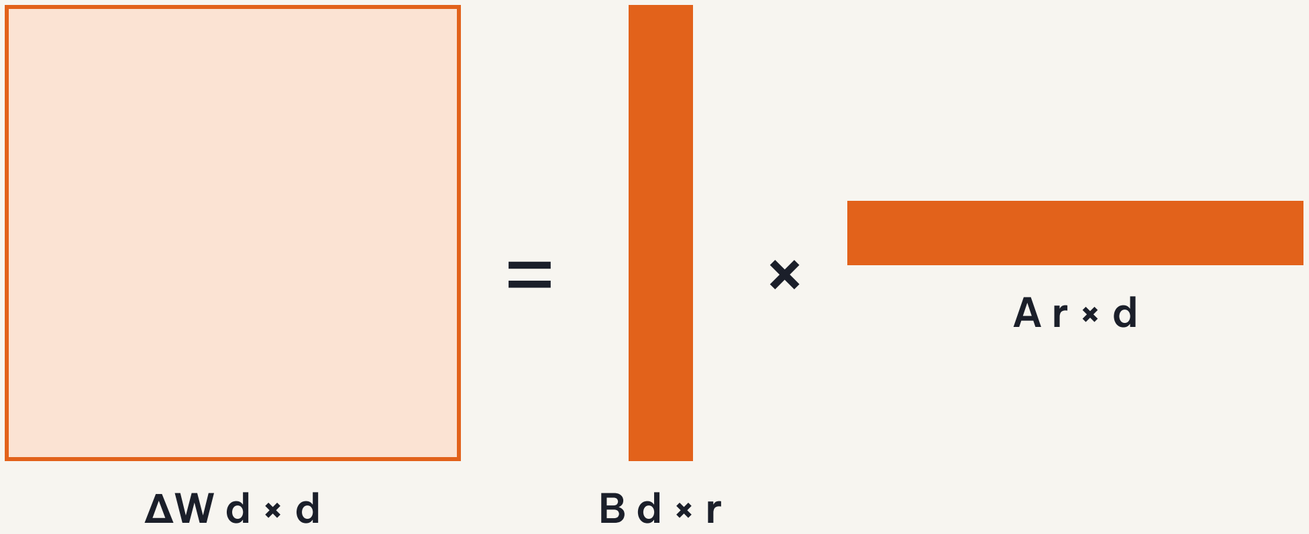
0,59 MB

file adapter, so với 328 MB của cả model

0 thêm

chi phí inference sau khi gộp bản cập nhật

Hạng thấp: ghép từ hai ma trận mỏng



Với d = 4096 và r = 8:

16.777.216 → 65.536 số

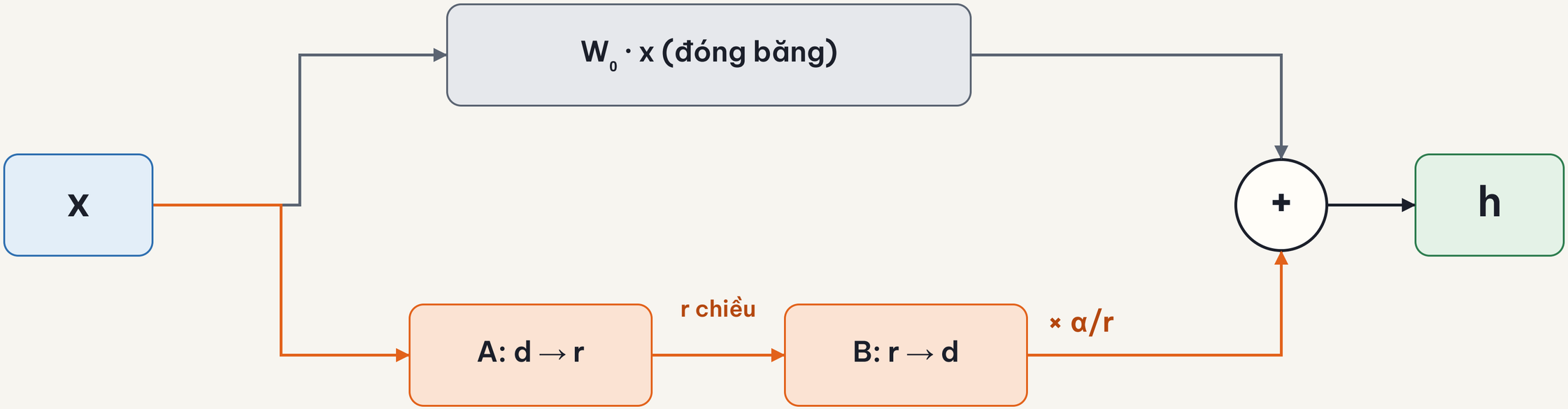
ít hơn 256×

Giả thuyết: phần thay đổi ΔW mà tác vụ mới cần có “hạng nội tại” thấp — model chỉ cần dịch chuyển theo vài hướng.

02 · LORA HOẠT ĐỘNG THẾ NÀO

Lượt forward có hai nhánh song song

$$h = W_0 x + (\alpha/r) \cdot B\,A\,x$$



Chỉ A và B nhận gradient. Ma trận ΔW đầy đủ d × d không bao giờ được tạo ra khi train.

02 · LORA HOẠT ĐỘNG THẾ NÀO

Ba chi tiết làm nên LoRA

B = 0

Bắt đầu từ model gốc

Ở bước 0, BA = 0 nên model y hệt model pre-train. A ngẫu nhiên để gradient chảy được; nếu cả hai bằng 0 thì không bao giờ học.

× α / r

Giữ độ lớn ổn định

Chia cho r giúp độ lớn bản cập nhật ổn định khi đổi r, nên không phải dò lại learning rate. Thường đặt α = r hoặc 2r.

$W' = W_0 + \Delta W$

Gộp sau khi train

W′ cùng kích thước W_0 nên inference nhanh y như model gốc. Hoặc giữ A, B riêng làm adapter có thể tháo lắp.

Hạng r phải đủ cho thay đổi thật

Thí nghiệm nhỏ: một lớp 128 × 128 "đã pre-train" phải học phần thay đổi ΔW có **hạng 4**. Cùng dữ liệu, 1.500 bước Adam.

r dưới 4 thì underfit. r = 4 bằng full fine-tuning nhưng **ít hơn 16×** tham số cần train.

Phương pháp	Tham số train	Loss cuối
Full fine-tuning	16.384	6,7e-9
LoRA r = 1	256	2,1e-2
LoRA r = 2	512	1,2e-2
LoRA r = 4	**1.024**	**9,3e-8**
LoRA r = 8	2.048	1,2e-6

03 · CÀI ĐẶT LORA

Một lớp LoRA bằng PyTorch thuần

```
class LoRALayer(nn.Module):
    def __init__(self, base, r=8, alpha=16):
        super().__init__()
        self.base = base
        for p in base.parameters():
            p.requires_grad_(False)   # đóng băng W0
        d_out, d_in = base.weight.shape
        self.scale = alpha / r
        self.A = nn.Parameter(torch.randn(r, d_in) * 0.01)
        self.B = nn.Parameter(torch.zeros(d_out, r)) # B = 0
    def forward(self, x):
        lora = (x @ self.A.T) @ self.B.T  # d → r → d
        return self.base(x) + self.scale * lora
    @torch.no_grad()
    def merge(self):
        self.base.weight += self.scale * self.B @ self.A
```

Bọc rồi đóng băng

Lớp gốc trở thành base; không trọng số nào của nó được train.

Hai ma trận mới

A ngẫu nhiên và nhỏ, B toàn số 0. Chỉ hai ma trận này được train.

Cộng thêm nhánh

base(x) cộng với output của nút cổ chai đã nhân hệ số.

Gộp để triển khai

Cộng ΔW vào W0 sau khi train xong.

03 · CÀI ĐẶT LORA

Gắn vào model thật rồi huấn luyện

```
def inject_lora(model, targets, r=8, alpha=16):
    for p in model.parameters():
        p.requires_grad_(False)      # đóng băng tất cả
    for parent in list(model.modules()):
        for name, child in list(parent.named_children()):
            if name in targets:      # thay bằng LoRA
                setattr(parent, name, LoRALayer(child, r, alpha))

model = AutoModelForCausalLM.from_pretrained("distilgpt2")
inject_lora(model, {"c_attn"})   # attention q, k, v của GPT-2
```

```
opt = torch.optim.AdamW([p for p in model.parameters()
                         if p.requires_grad], lr=2e-3)
loss = model(**batch).loss   # nhãn = -100 ở phần prompt
loss.backward(); opt.step()
```

Chọn lớp mục tiêu

Ở đây là lớp chiếu attention c_attn. Thực tế: mọi lớp tuyến tính.

Learning rate cao hơn

1e-4 đến 2e-3, khoảng 10× full fine-tuning.

Che phần prompt

Chỉ tính loss trên token câu trả lời, để model học trả lời chứ không học đặt câu hỏi.

03 · CÀI ĐẶT LORA

Trước và sau: dạy về một hành tinh bịa

12 cặp hỏi–đáp về hành tinh hư cấu “Zorblax”, 40 epoch, khoảng 22 giây trên Apple M4.

DISTILGPT-2 GỐC

Q: What is the capital of Zorblax?

A: Zorblax is a capital of Zorblax. It is a capital of Zorblax...

CÓ ADAPTER LORA

Q: What is the capital of Zorblax?

A: The capital of Zorblax is Glimmerton.

147.456
tham số được train trên 82 triệu (0,18%)

0,59 MB
file adapter, so với 328 MB của cả model

Tắt adapter
là model gốc quay lại nguyên vẹn

Thực tế: dùng thư viện peft

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=8, lora_alpha=16, lora_dropout=0.05,
    target_modules=["c_attn"],
    fan_in_fan_out=True,   # Conv1D của GPT-2
    task_type="CAUSAL_LM",
)
model = get_peft_model(model, config)
model.print_trainable_parameters()
# trainable params: 147,456 || trainable%: 0.1797

model.save_pretrained("zorblax_adapter")
merged = model.merge_and_unload()
```

Code tự viết	peft
LoRALayer(r, alpha)	LoraConfig
inject_lora	get_peft_model
count_params	print_trainable_parameters
enabled = False	disable_adapter()
lưu A và B	save_pretrained
merge()	merge_and_unload()

Cùng số tham số train với bản tự viết, đường loss gần như trùng nhau: đó là cùng một thuật toán.

ÁP DỤNG THỰC TẾ

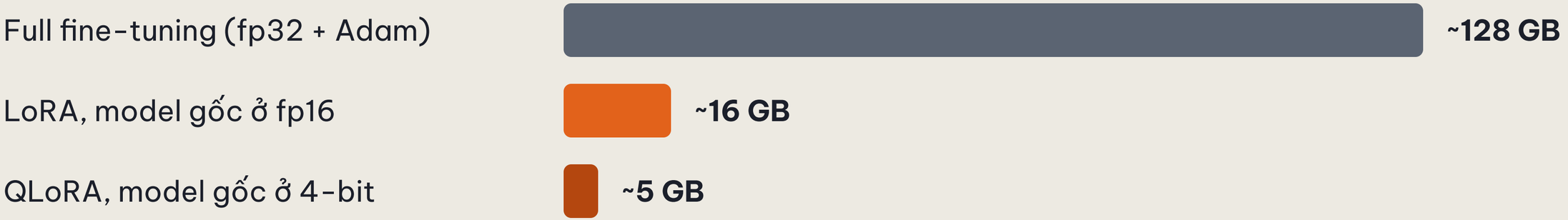
Siêu tham số khởi đầu hợp lý

Tham số	Bắt đầu với	Vì sao
r	8–16	Ít dữ liệu cần ít sức chứa; 32–64 cho tác vụ khó
lora_alpha	r hoặc 2r	Giữ α / r cố định khi đổi r
target_modules	mọi lớp tuyến tính	Chất lượng tốt hơn chỉ dùng q, v của attention
learning rate	1e-4 – 2e-4	Cao hơn full fine-tuning khoảng 10×
lora_dropout	0 – 0,1	Tăng lên khi dữ liệu ít
epochs	1–3	Nhiều epoch hơn dễ overfit

ÁP DỤNG THỰC TẾ

QLoRA: fine-tune model lớn trên 1 GPU

W_0 đã đóng băng nên không cần độ chính xác cao: lưu nó ở 4-bit (NF4), còn A và B vẫn train ở bf16. Bộ nhớ cho model 8B, chưa tính activation:



QLoRA: Dettmers và cộng sự, 2023. Cần GPU NVIDIA và thư viện bitsandbytes. Các thanh được vẽ đúng tỉ lệ.

ÁP DỤNG THỰC TẾ

Bốn lỗi hay gặp

Quên đóng băng model

Thực chất bạn đang full fine-tune. Luôn kiểm tra print_trainable_parameters().

Learning rate quá thấp

LR của full fine-tuning như 2e-5 gần như không làm LoRA nhúc nhích. Bắt đầu quanh 1e-4 đến 2e-4.

Sai chat template

Model instruct có định dạng hội thoại riêng. Dùng tokenizer.apply_chat_template.

Kỳ vọng dạy kiến thức mới

LoRA mạnh nhất ở định dạng, văn phong và tác vụ. Với khối lượng kiến thức lớn, hãy kết hợp với RAG.

TÓM TẮT

LoRA trong bốn dòng

1. Đóng băng W_0, chỉ học $\Delta W = (\alpha/r)\cdot BA$ với hạng r nhỏ.
2. Chỉ train khoảng 0,1–1% tham số; adapter chỉ vài MB.
3. Khởi tạo B = 0; gộp sau khi train để không tốn thêm độ trễ.
4. Viết được trong 15 dòng PyTorch; peft làm đúng điều đó.

Notebook: lora_finetune_scratch.ipynb · Bài báo: arxiv.org/abs/2106.09685 · Tài liệu: huggingface.co/docs/peft